Department of Engineering Science
School of Science & Technology



CES 552 Network Architecture
Transport Layer Part I

Dr. Jingxian Wu
(707) 664-3491
Jingxian.wu@sonoma.edu

OUTLINE

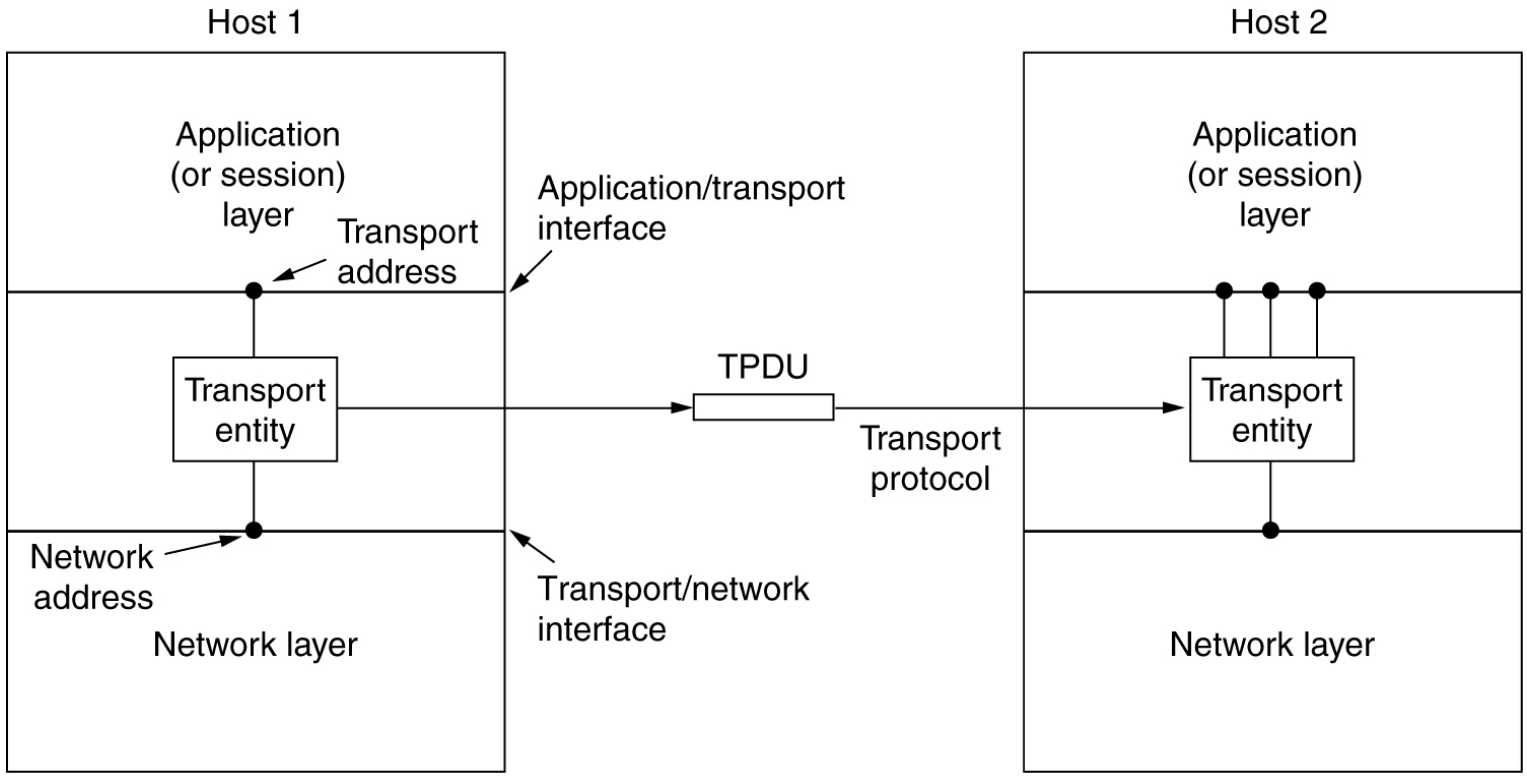
- **Functions & Services**
- **Elements of transport protocol**
- **A simple transport protocol**
- **Internet transport protocol: UDP**
- **Internet transport protocol: TCP**
- **Performance issues**

TRANSPORT LAYER: FUNCTIONS

- **Provide reliable, cost-effective data transport from source to destination.**
  - Independent of the physical network
  - Independent of the network structure & topology
  - The first true end-to-end layer
    - Transport layer doesn't exist on routers.
    - Transport layer on the source is directly talking with transport layer on the destination.
    - On other layers, e.g. network layer, the conversation is usually between intermediate neighbors.

TRANSPORT LAYER: FUNCTIONS

- **Transport entity: software and/or hardware that implement the transport functions**
  - Could be part of the operating system (OS) kernel
  - Or a separate library package



TRANSPORT LAYER: SERVICES

- **Connection-oriented service**
  - Three phases:
    - Connection establishment
    - Data transfer
    - Connection release
  - Packets belonging to the same connection might take different route!
    - If the underlying network layer is connectionless.
  - Example: TCP (Transport Control Protocol)
- **Connectionless service**
  - No connection required
  - E.g. UDP (User Datagram protocol)
  - Could be on top of connection-oriented network layer
    - E.g. UDP on top of ATM

TRANSPORT LAYER: WHY TRANSPORT LAYER?

- **Network layer providing connectionless and connection oriented services as well.**
  - Why do we need another layer?
- **Network layer mainly resides on routers**
  - Routers are owned by operators
  - Users have no control about the operation of the network layer.
  - If something goes wrong in the network, the users could do nothing to stop it.
- **We need an addition layer on top of network layer to improve the QoS.**
  - If error happens in network, simply set up another connection.
  - Lost data can be detected and compensated by transport layer.
  - Provide a unified interface to users
    - Application programmers do not need to worry about the underlying network

TRANSPORT LAYER: SERVICE PRIMITIVES

- **Service primitives**
  - A set of interfaces for the user to access the service
  - Usually in the form of function calls.
  - Analogy: printf( ) is the service primitive for the format printing service provided by C language.
    - Users do not need to know the implementation of the function.
- **Different transport layer protocols have different service primitives**

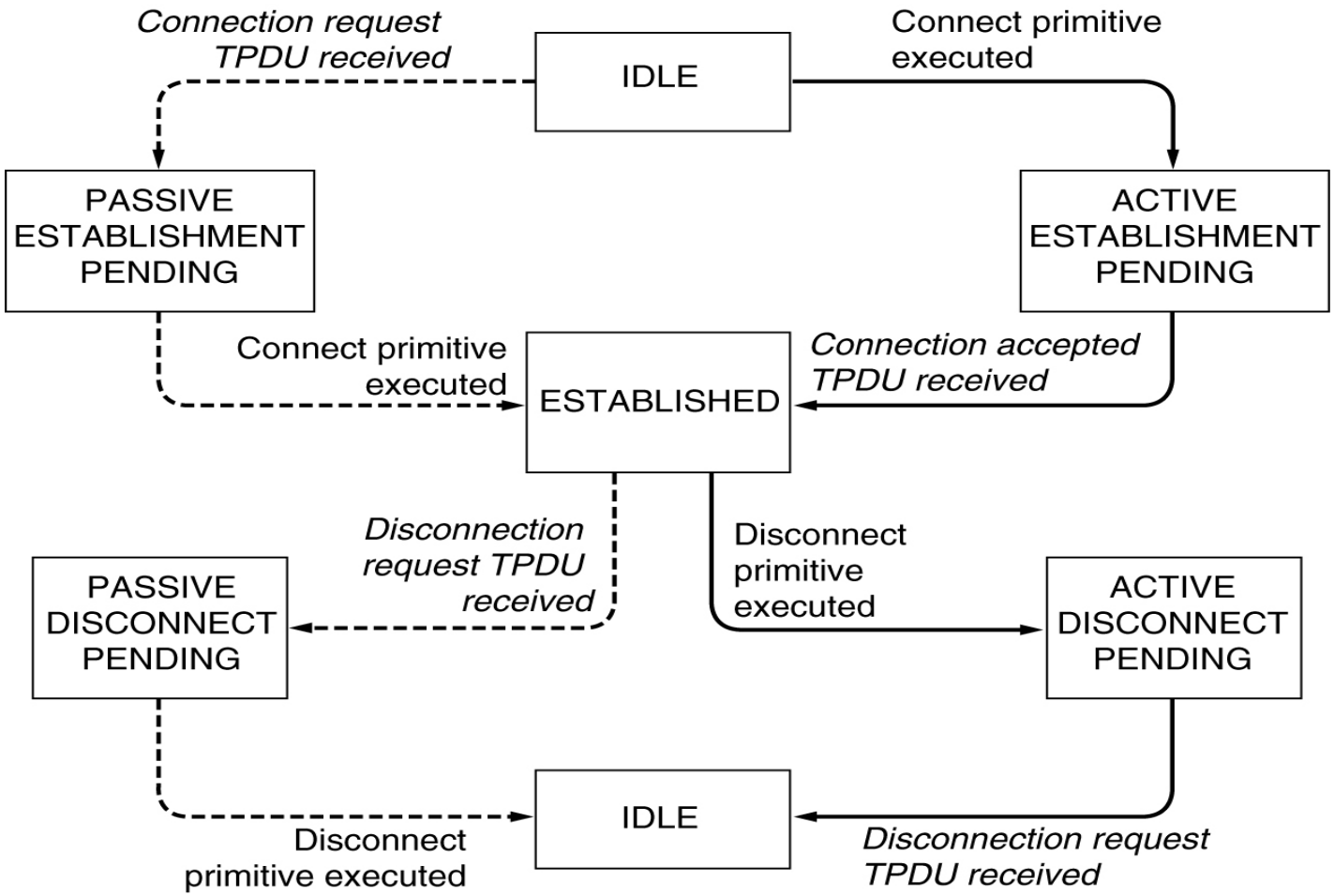
TRANSPORT LAYER: SERVICE PRIMITIVES

- **A simple example**

Primitive	Packet sent	Meaning
LISTEN	(none)	Block until some process tries to connect
CONNECT	CONNECTION REQ.	Actively attempt to establish a connection
SEND	DATA	Send information
RECEIVE	(none)	Block until a DATA packet arrives
DISCONNECT	DISCONNECTION REQ.	This side wants to release the connection

- **The service primitives shield the underlying implementations from users**
  - Acknowledgement, lost packets, congestion, etc. are invisible to users.
  - Provide reliable service on top of unreliable networks
  - All users need to do is call SEND to send a packet at source, and call RECEIVE at destination to retrieve the packet.
- **Used directly by programmers or users**
  - Usually are convenient and easy to use.

TRANSPORT LAYER: A SIMPLE CONNECTION



TRANSPORT LAYER: BERKELEY SOCKETS

- **Berleley sockets**
  - Socket: access point for the service in transport layer.
  - Service primitives used in Berkeley Unix for TCP.
  - Part of the OS kernel.
  - The most widely used service primitives for TCP.

Primitive	Meaning
SOCKET	Create a new communication end point
BIND	Attach a local address to a socket
LISTEN	Announce willingness to accept connections; give queue size
ACCEPT	Block the caller until a connection attempt arrives
CONNECT	Actively attempt to establish a connection
SEND	Send some data over the connection
RECEIVE	Receive some data from the connection
CLOSE	Release the connection

In Linux or Unix system, use *man command_name* (*e.g. man socket*) to get more information.

TRANSPORT LAYER: BERKELEY SOCKETS

- **sock_ID = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP)**
  - Create a new socket in transport layer. Each transport layer socket provides services to one user.
  - AF_INET: address domain (two hosts on the Internet)
  - SOCK_STREAM: type of socket (reliable byte stream)
  - IPPROTO_TCP: the protocol to be used (TCP)
  - Returns an ID to the newly created socket.
- **bind(sock_ID, (struct sockaddr *)& address, sizeof(address))**
  - Bind a network address to the newly created socket (sock_ID).
  - struct sockaddr contains three fields, including
    - The type of address (IP address or others)
    - IP address
    - TCP port number

TRANSPORT LAYER: BERLELEY SOCKETS

- **listen(sock_ID, QUEUE_SIZE)**
  - Allows the program listening on the socket for connections. Usually executed by the server.
  - Sock_ID: the ID of the socket to be listened.
  - QUEUE_SIZE: a constant specifying the queue size, i.e., how many connections can be waiting while the process is handling another connection.

TRANSPORT LAYER: BERKELEY SOCKET

- **sa = accept(sock_ID, (struct sockaddr *)& client_addr, sizeof(client_addr))**
  - After this function is executed, the program will be blocked for connection request. (e.g., the system will not do anything until a connection request is received)
  - client_addr: the IP address and port number from the requesting hosts (client)
  - Sizeof(client_addr): the size of the address
  - Usually executed on the server side.
  - If success, return the descriptor of this connection.
- **ca = connect(csocket_ID, &server_addr, sizeof(server_addr))**
  - Sends a connection request to server
  - csocket_ID: the ID of the socket on the client side.
  - server_addr: the (IP address, port #) of the server
  - sizeof(servier_addr): size of the server_addr.
  - Will return the descriptor of the successful connection.
  - Usually executed on the client side.

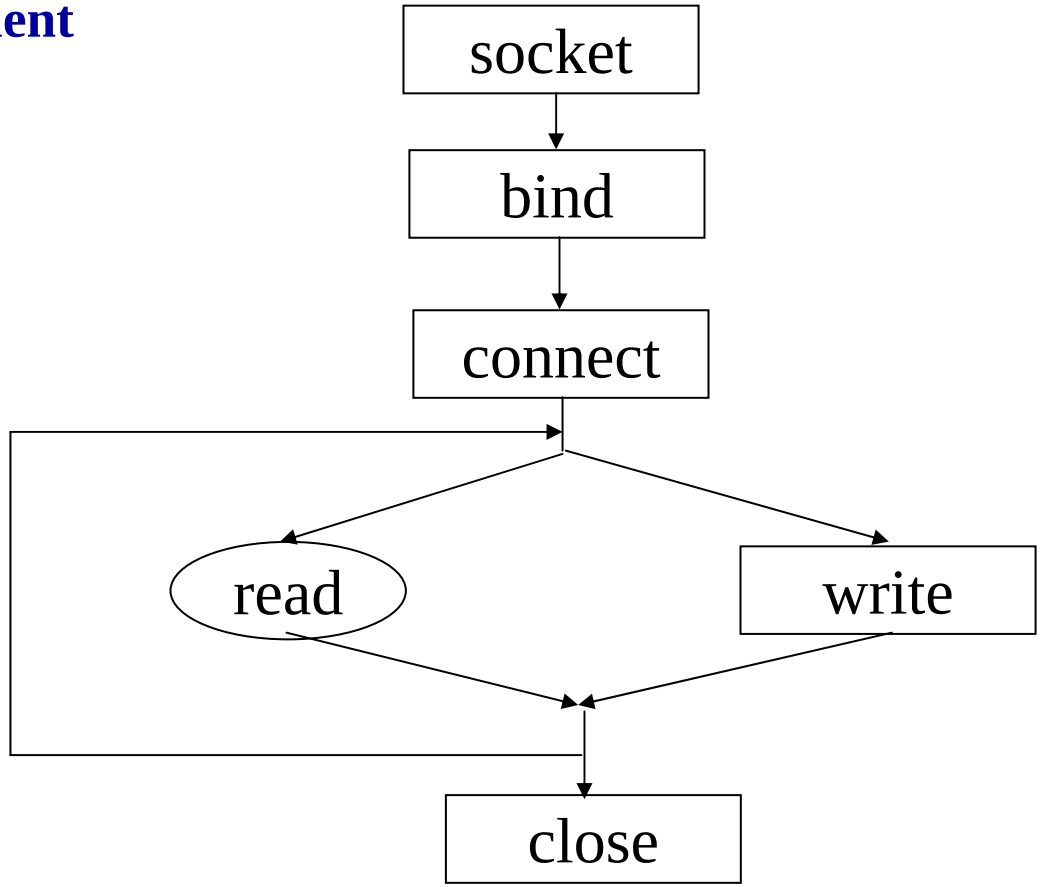
TRANSPORT LAYER: BERKELEY SOCKET

- **write(ca, buf, BUF_SIZE)**
  - Send data to the connection specified by the descriptor ca.
  - buf: pointer pointing to the data to be sent
  - BUF_SIZE: how many data to be transferred
- **read(sa, buf, BUF_SIZE)**
  - read data from connection specified by the descriptor sa.
  - buf: pointer pointing to the buffer to store the data
  - BUF_SIZE: how many data to be read from sa.
- **close(ca), close(sa), close(socket_ID)**
  - Close the socket and the connections

A quick tutorial:
http://www.cs.rpi.edu/courses/sysprog/sockets/sock.html

TRANSPORT LAYER: BERKELEY SOCKET

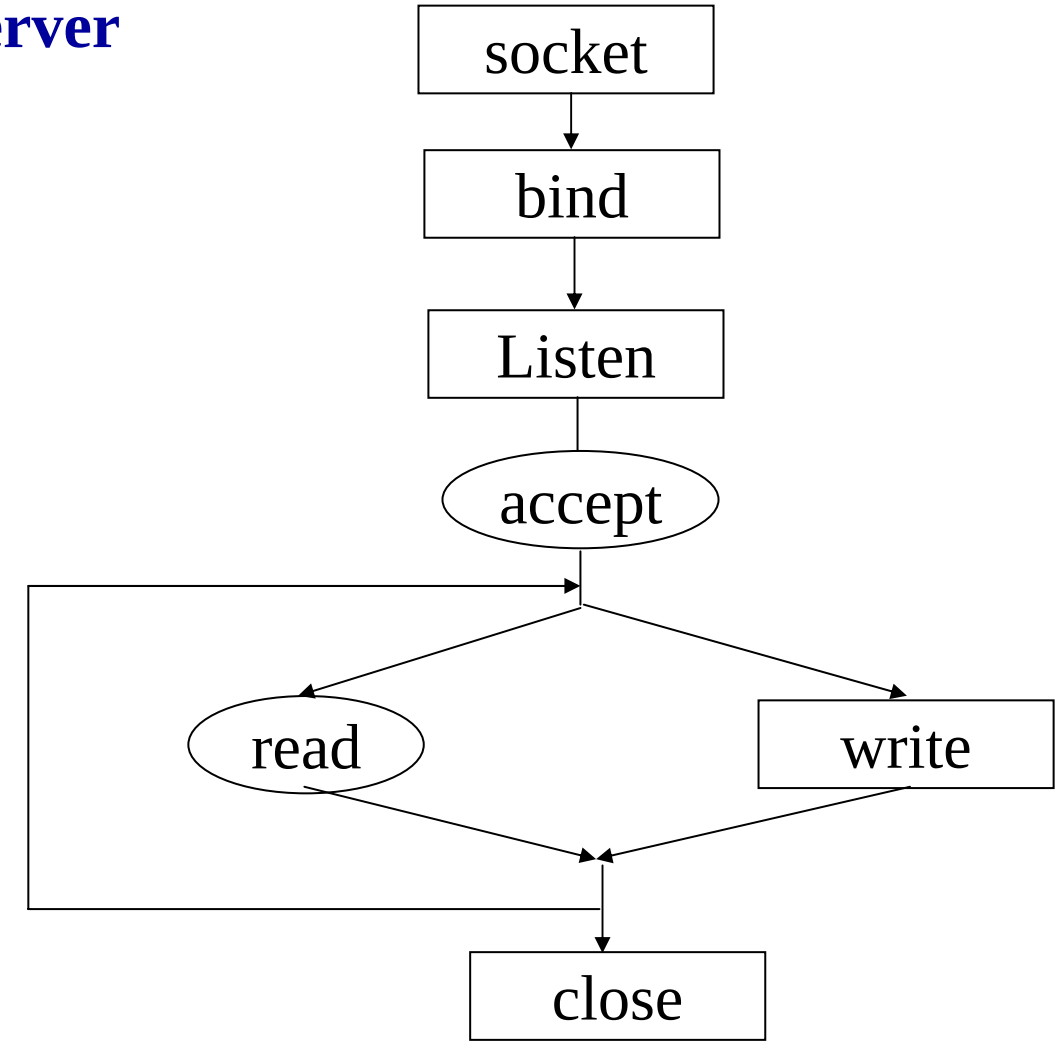
- **Client**



read :System will be blocked until data are available for reading

TRANSPORT LAYER: BERKELEY SOCKET

- **server**

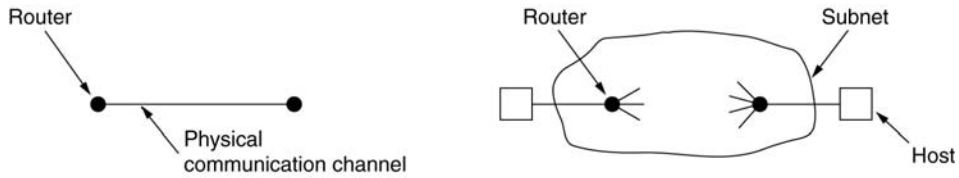


OUTLINE

- Functions & Services
- **Elements of transport protocol**
- A simple transport protocol
- Internet transport protocol: UDP
- Internet transport protocol: TCP
- Performance issues

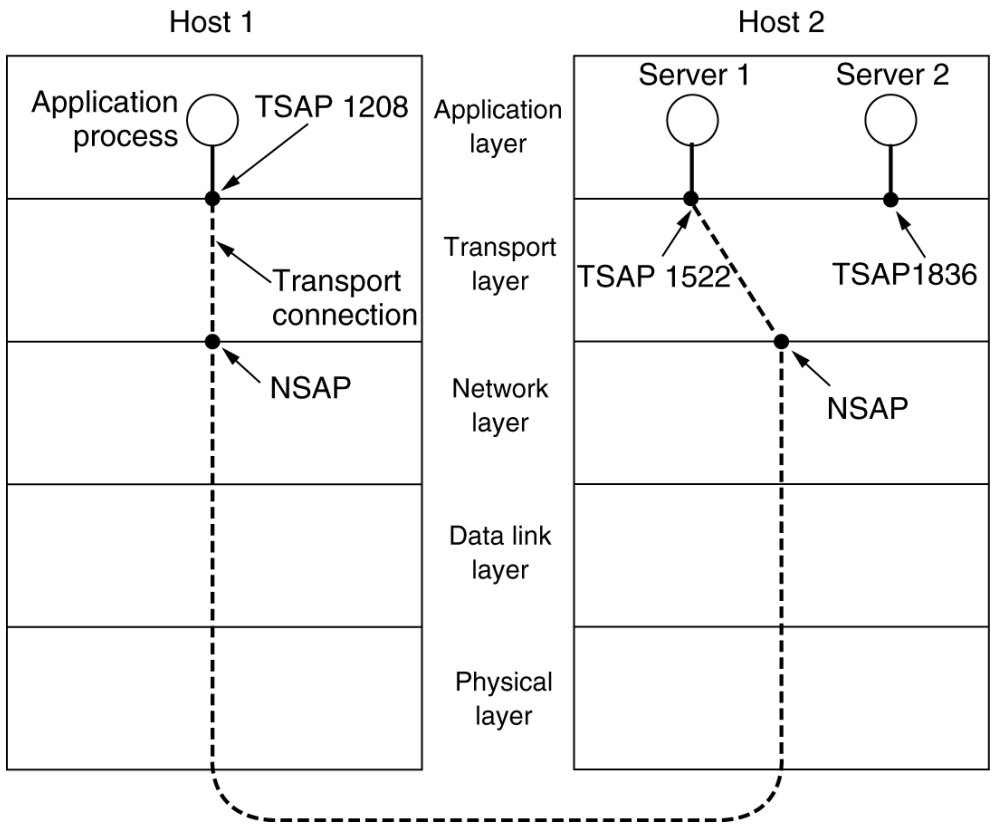
ELEMENTS

- **Addressing**
  - The address of the host at the other end
- **Connection establishment**
  - Much complicated than that of data link layer



- **Connection release**
- **Flow control & Buffering**
  - Data link layer: point to point line
  - Transport layer: the packets might be hiding somewhere in the subnet
  - Transport layer: each transport layer supports many simultaneous connections
- **Crash recovery**

ELEMENTS: ADDRESSING



- **TSAP (Transport service access point)**
  - TCP and UDP: port #.
- **NSAP (network service access point)**
  - IP: IP address

ELEMENTS: ADDRESSING

- **How could a client know the port number of a server application?**
  - Well known applications get fixed port numbers
    - E.g. TCP port 0 ~ 1023 are assigned to well known applications.
      - ftp: 21, ssh: 22, telent: 23, smtp: 25, http: 80, etc.
  - Application can be registered to get a fixed port
    - A complete list of TCP/UDP port assignment can be found at

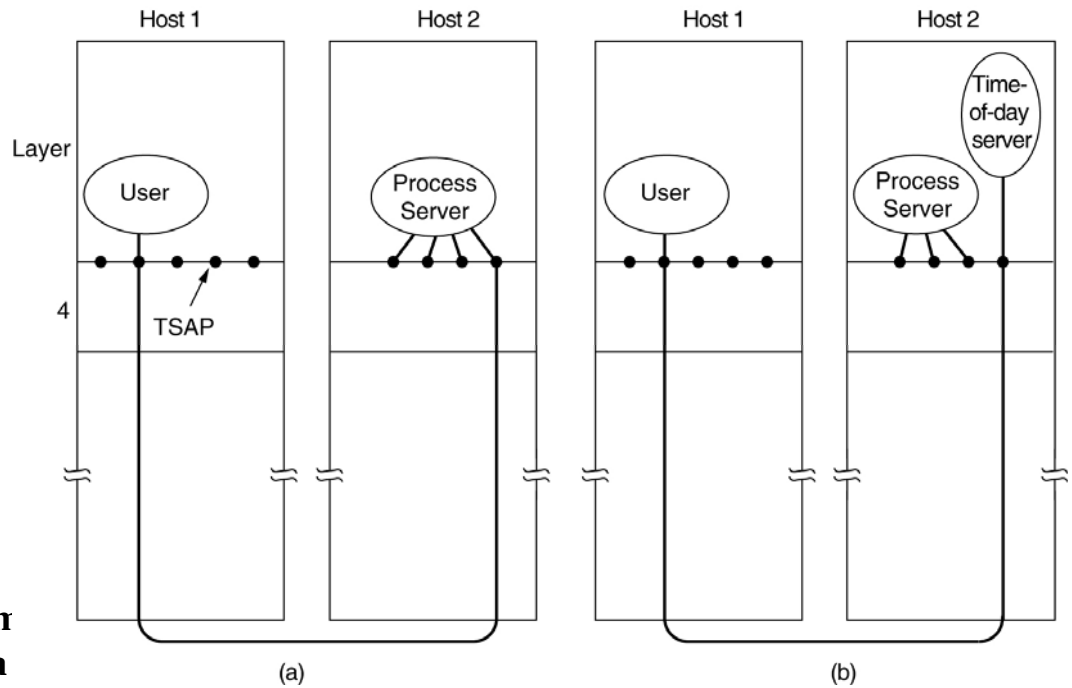
    http://www.iana.org/assignments/port-numbers
    - For TCP/UDP, port # 1024 ~ 49151 are for registered applications
  - Dynamic port
    - TCP/UDP, port # 49152 ~ 65535

ELEMENTS: ADDRESSING

- **Address assignment: directory server**
  - One directory server is running on a well known port.
  - Directory server contains the mapping between (service name, port #) for other services running on the computer.
  - When a new service is created, it must register itself with the directory server.
  - Client connects to the directory server first to find out the port # of the service it wants to connect.
  - Analogy: phone directory service
    - Given the name to the operator, the operator gives you the corresponding phone #.
  - E.g. portmapper of SUN RPC (Remote procedure call) (port # 111)
    - Lists the port number of all RPC services (NFS)

ELEMENTS: ADDRESSING

- **Initial connection protocol**
  - A special process server is used to listen to a series of ports.
  - When there is a connection request on a given port, the process server will spawns the requested server (ftp, telnet, etc.) corresponding to the particular port.
  - By this means, only one server program (the process server) needs to be running all the time, and system resources will be saved.



ELEMENTS: CONNECTION ESTABLISHMENT

- **Seems quite easy**
  - One transport entity sends a CONNECTION REQUEST
  - The other transport entity replies with CONNECTION ACCEPTED
- **Potential problems:**
  - The packet(s) can get lost in subnet.
  - The packet(s) might be “stored” in the network.
    - Due to congestion, the packet is delayed for a considerable amount of time in the network
    - After a while, the packet shows up.
    - Unique for transport layer.
  - Duplicate packets
    - The acknowledgment is delayed due to congestion.
    - Sender times out, retransmission.

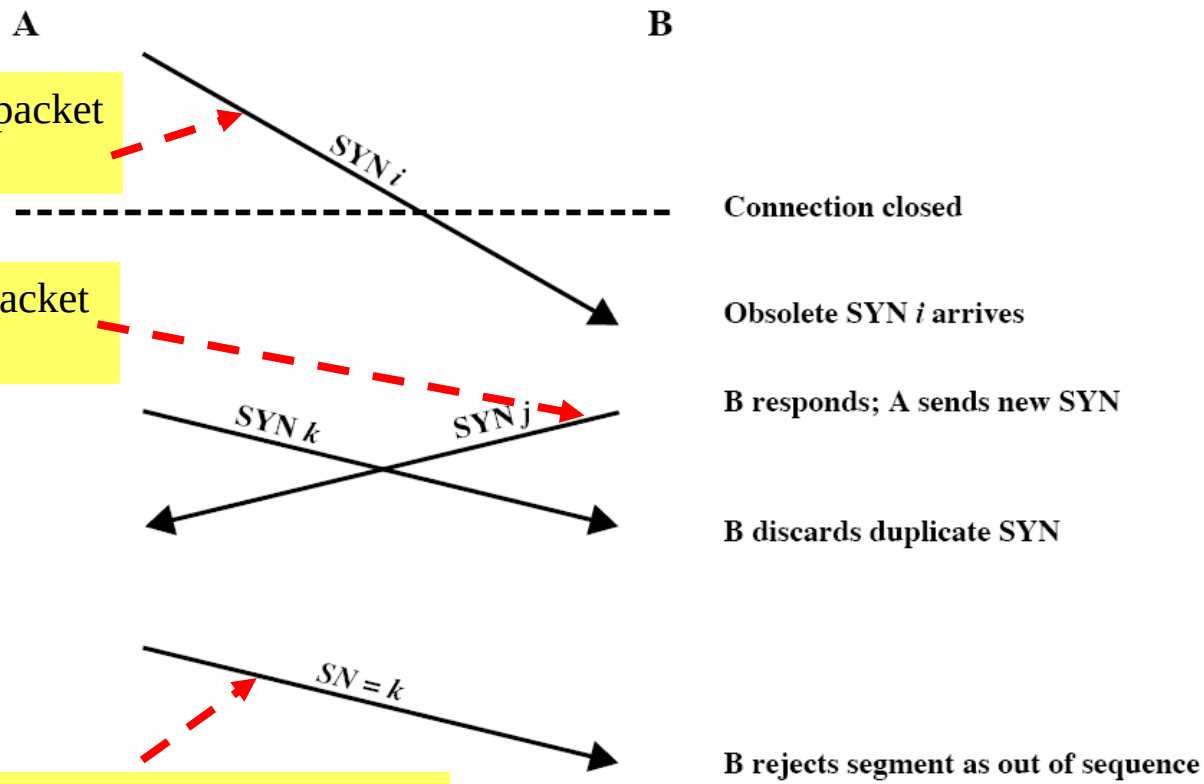
major problem: delayed duplicate

ELEMENTS: CONNECTION ESTABLISHMENT

- **Solutions to delayed duplicate**
  - Restrict the maximum lifetime of a packet to be T_0
    - If the packet's age is larger T_0, it will be discarded.
    - If we wait a time $T=kT_0$ after the packet is transmitted, where k is a small integer, then we can make sure that both the original packet and its acknowledgement is dead.
  - Assign each packet a unique sequence number.
    - The range of sequence number is large enough to avoid wrap around. (32-bits is used in TCP)
    - What if the system crashes?
      - Assign the sequence # based on some reference that are not going to be affected by crash
        - » E.g. based on the computer clock (after computer crashes, the computer clock is still running).

ELEMENTS: CONNECTION ESTABLISHMENT

- **Two-way hand-shake**



connection request, the first data packet from A will have a seq # of i

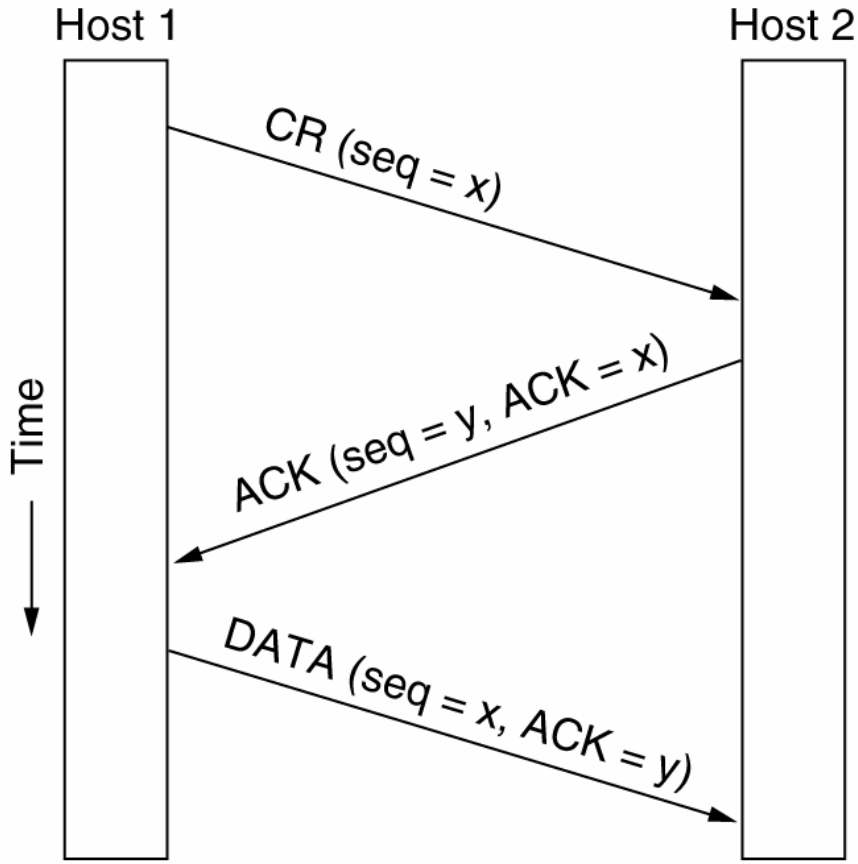
connection accept, the first data packet From B will have a seq # of j

B is expecting a packet with seq. # i
Thus this packet will be rejected as out of sequence

Two-way hand-shake is not suitable for transport connection establishment!

ELEMENTS: CONNECTION ESTABLISHMENT

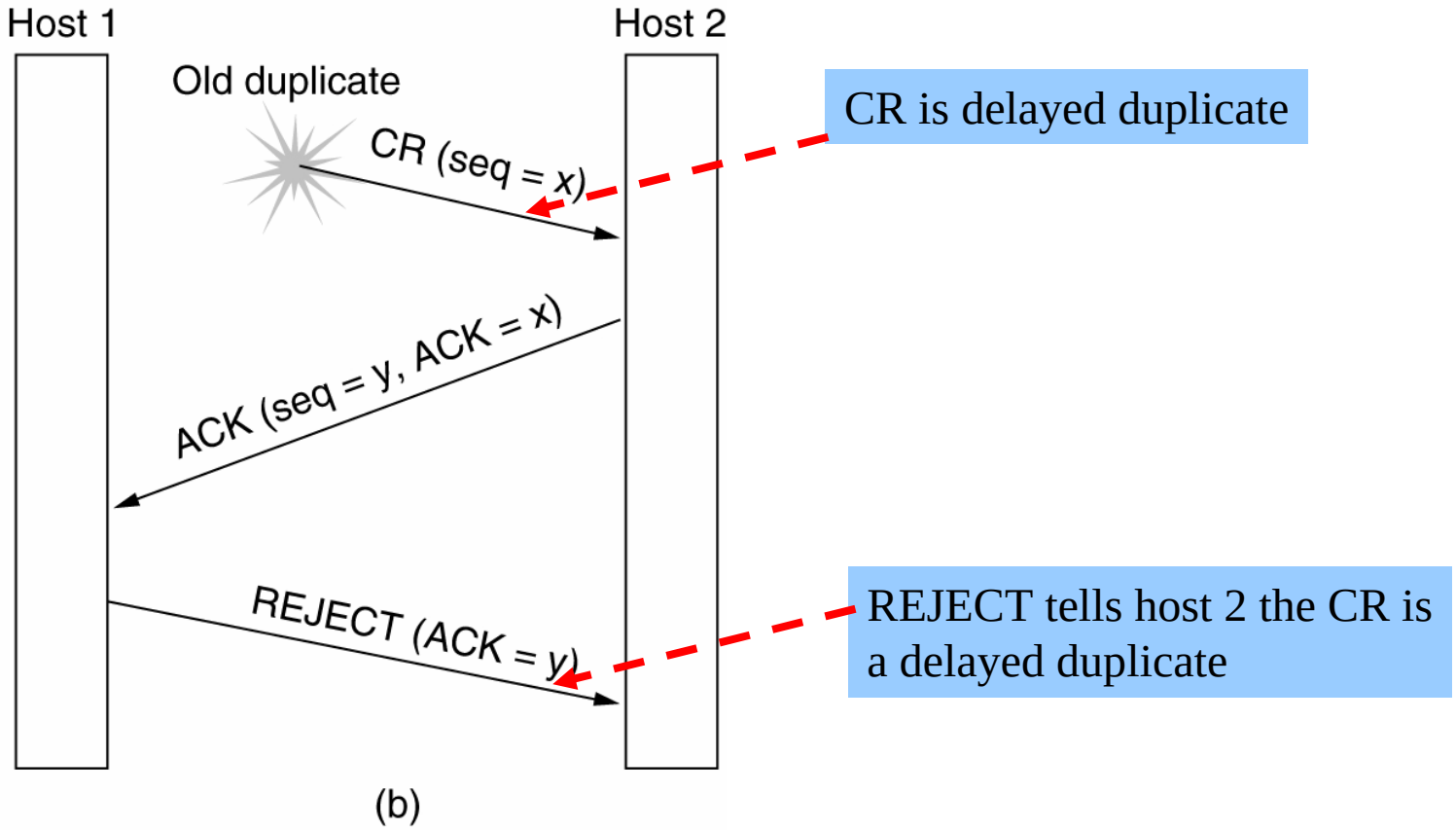
- **Three-way hand-shake**



(a)

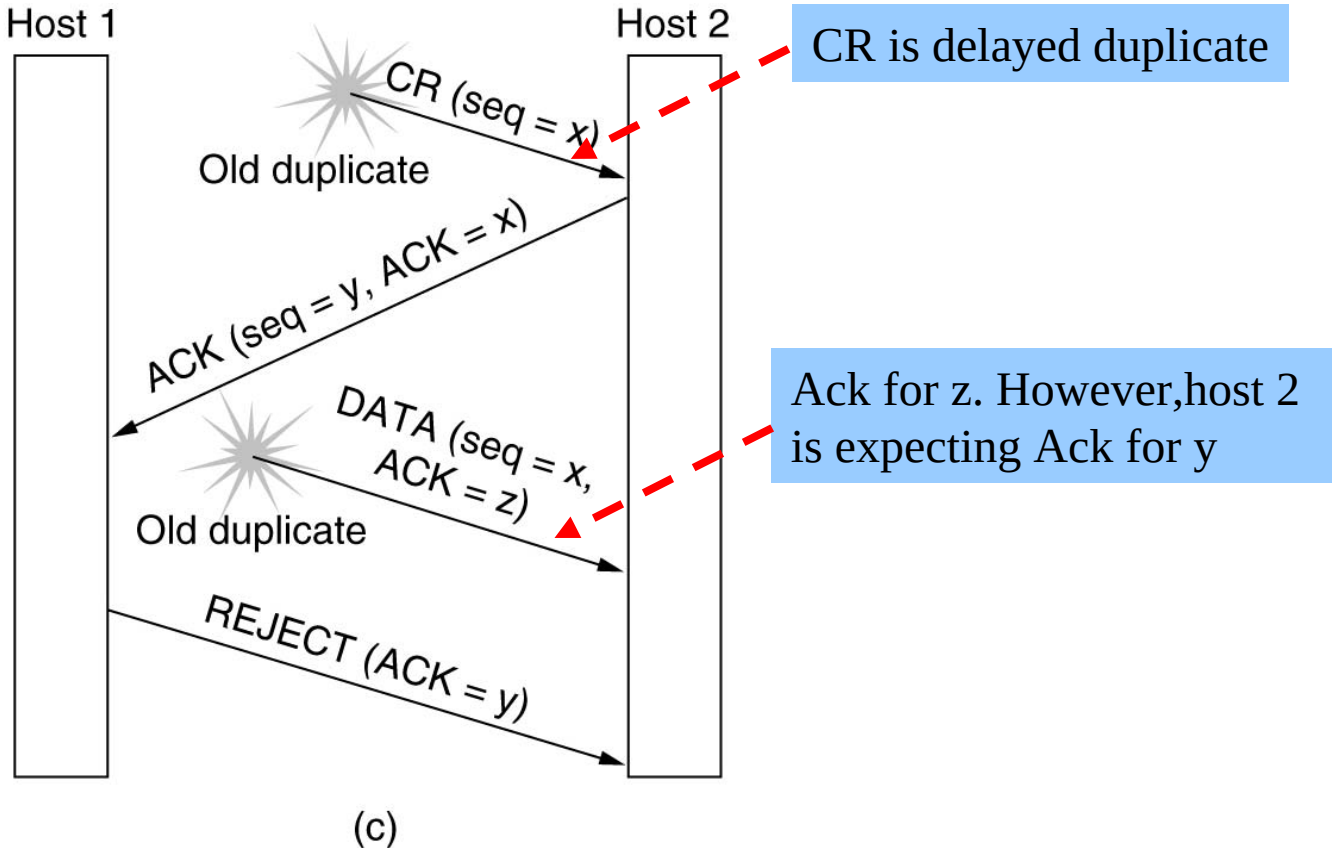
ELEMENTS: CONNECTION ESTABLISHMENT

- **Three-way hand-shake: delayed duplicate connection request**



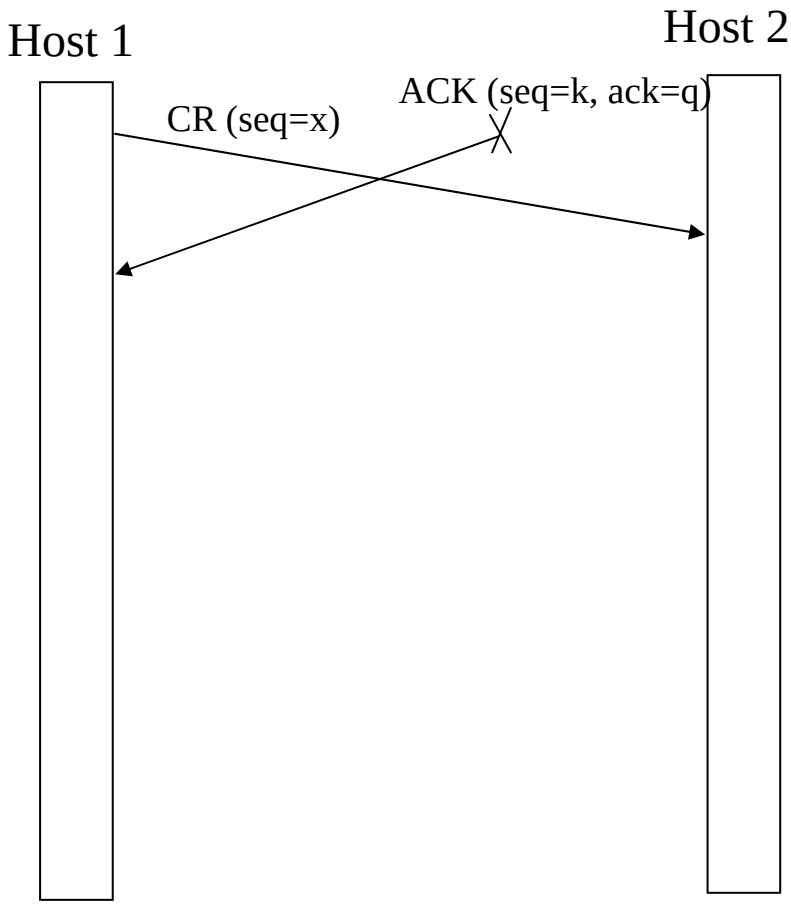
ELEMENTS: CONNECTION ESTABLISHMENT

- **Three-way hand-shake: both CR and ACK are delayed duplicate**



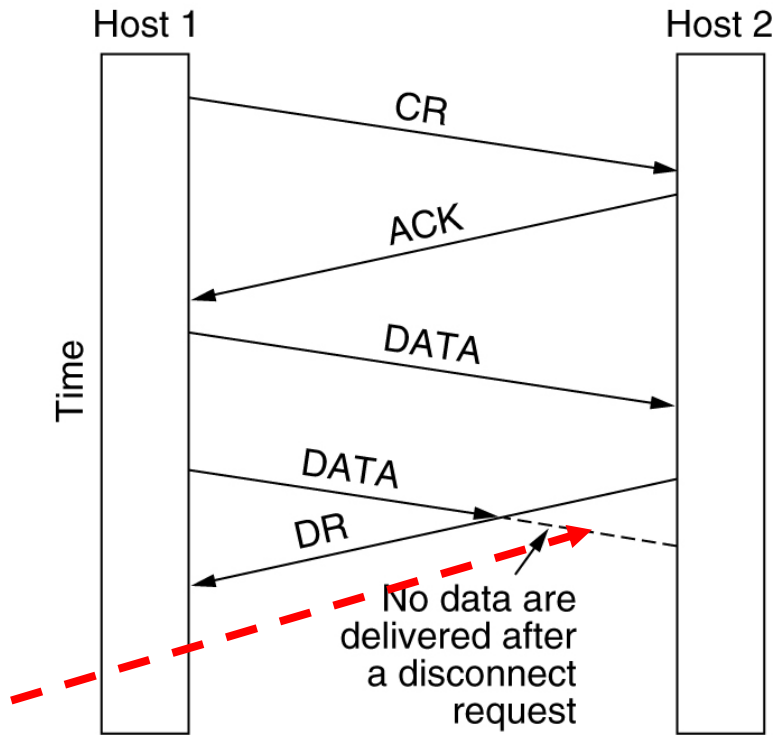
ELEMENTS: CONNECTION ESTABLISHMENT

- **Example**



ELEMENTS: CONNECTION RELEASE

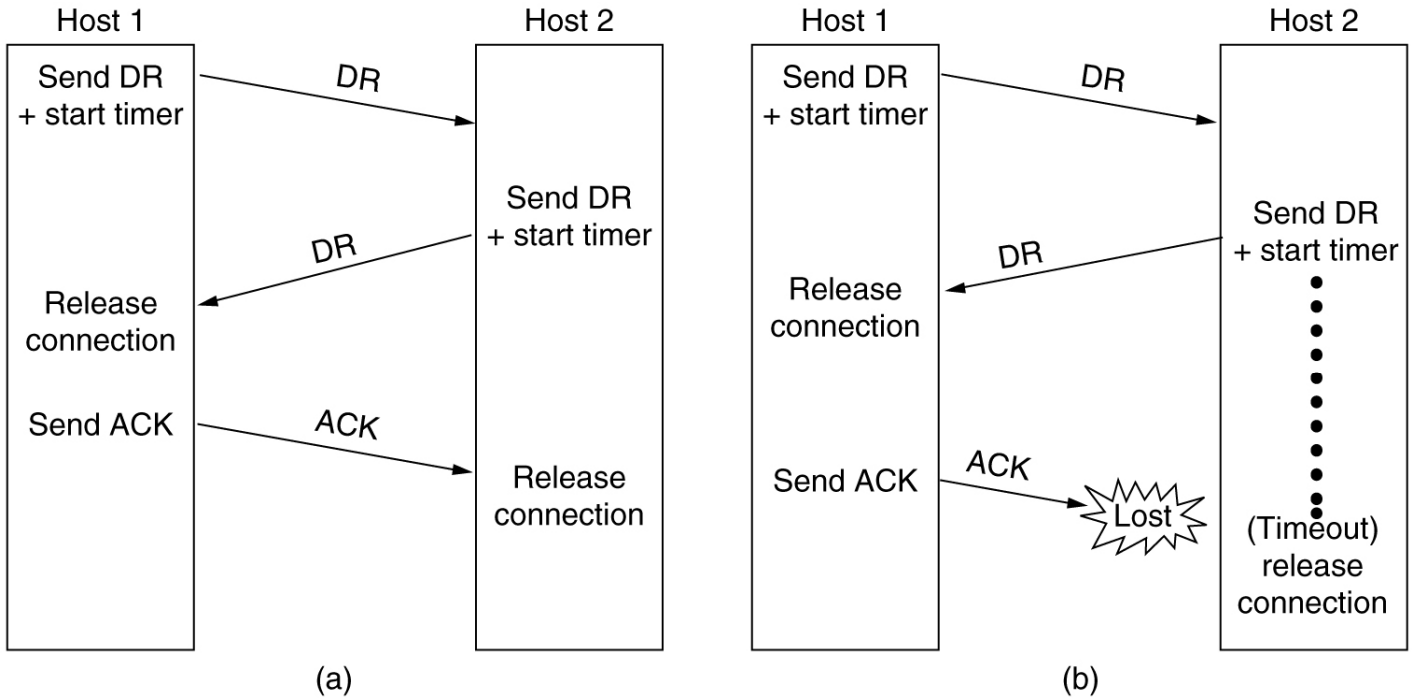
- **Asymmetric release**
  - When one party disconnects, the connection is released
  - Analogy: telephone system
  - Potential problem: data loss



Data arrives after disconnect request (DR) will be discarded!

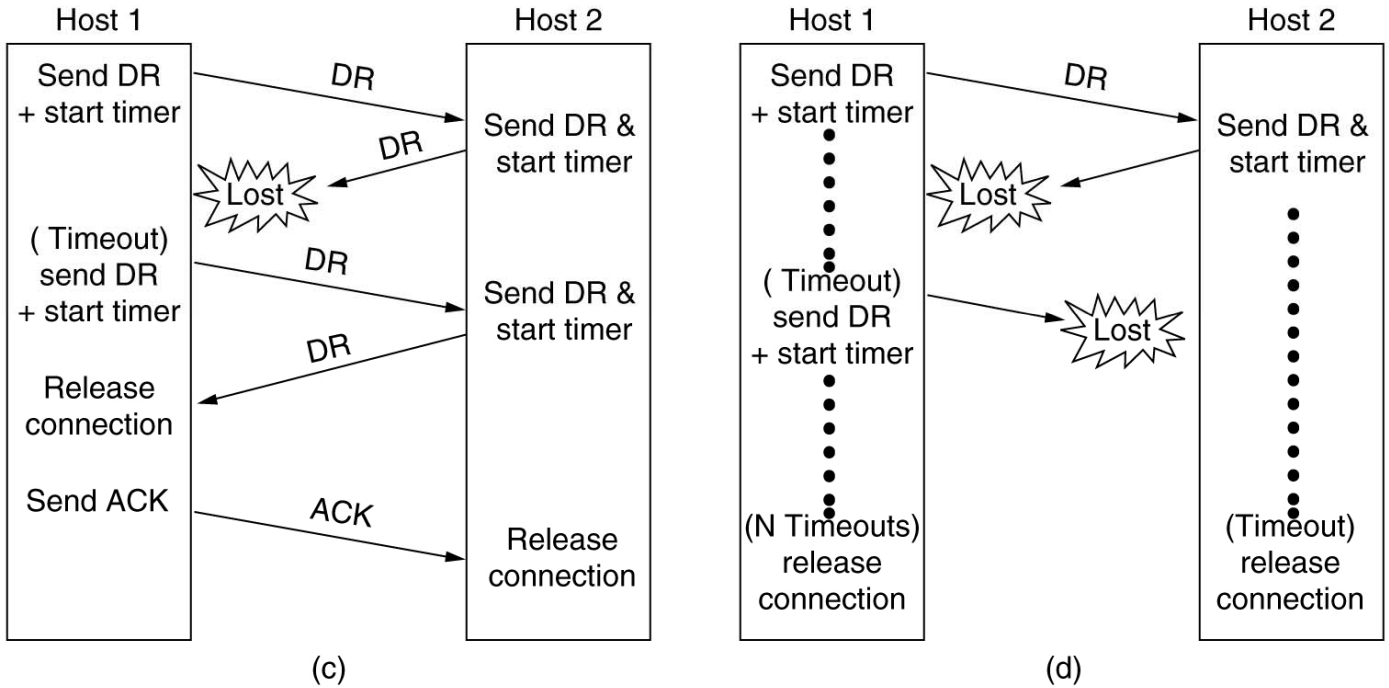
ELEMENTS: CONNECTION RELEASE

- **Symmetric release**
  - The two communication directions are released independently.
  - After a Disconnect Request is issued, the host will still accept data.
  - Only after both hosts issue Disconnect Request will the connection be released.



ELEMENTS: CONNECTION RELEASE

- **Symmetric release**



- **What if all the DRs from host 1 are lost?**
  - Host 1 time out, retry N times, release connection
  - Host 2 doesn't know about DR at all.
    - If there is no activity from Host 1 for a while, host 2 release connection.

ELEMENTS: FLOW CONTROL AND BUFFERING

- **Flow control**
  - Keep fast transmitter from swamping slow receiver.
  - Fast transmitter & slow receiver ➔ buffer overflow at receiver.

- **Differences from flow control at data link layer**
  - Transmission delay between peer transport entities is long.
    - Long delay for flow control information.
  - Transmission delay is highly variable.
    - A network is between the peer entities.
    - Difficult to effectively use a timeout mechanism for retransmission.

ELEMENTS: FLOW CONTROL AND BUFFERING

- **Flow control method 1: when Rx buffer is full, transport layer refuses more packets from network layer**
  - Packets being discarded at network layer.
  - Trigger the flow control or congestion control at lower layer.
  - This is not desirable. Why?
    - E.g. two machines have two transport connections at the same time.
    - These two connections are using the same network layer connection.
    - Only one of the two transport layer connections needs flow control.
    - However, network layer could tell the differences between the two ➔The traffic from both connections will be regulated.

ELEMENTS: FLOW CONTROL AND BUFFERING

- **Flow control method 2: use sliding window method as in data link layer.**
  - Any any instant, the sender is allowed to send up to *m* packets without being acknowledged
    - E.g. $m = 5$

    Sender sends out 0 1 2 3 4 (0 1 2 3 4 are not Ack-ed)

    Sender receives the Ack for 0 (1 2 3 4 are not Ack-ed)

    Sender sends out 5 (1 2 3 4 5 are not Ack-ed)

    Sender receives the Ack for 3 (4 5 are not Ack-ed)

    Sender sends out 6 7 8 (4 5 6 7 8 are not Ack-ed)

    0 1 2 3 4 5 6 7 8
  - If the Rx buffer is full, the Rx doesn't send back Ack.
    - The traffic from Tx is regulated by Rx.

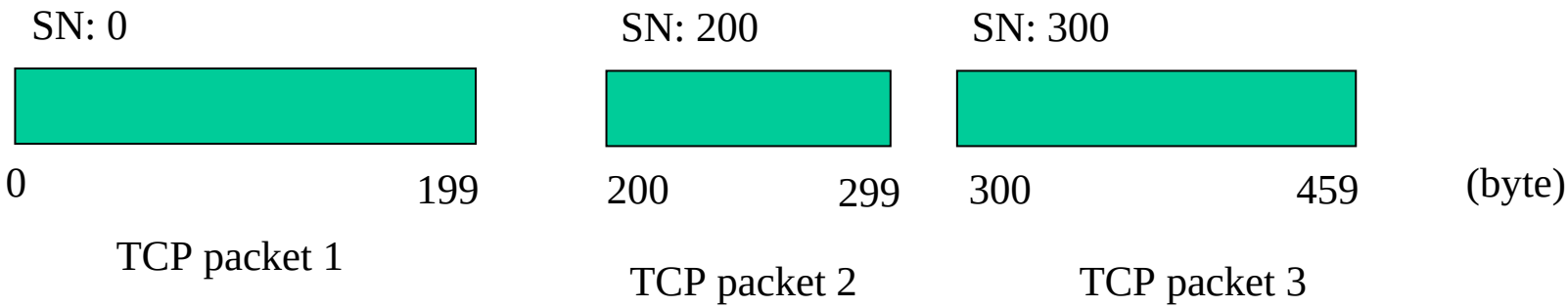
ONE Ack# for TWO things: 1. all packets up to the Ack# have been received
2. sender can send more packets based on Ack#.

ELEMENTS: FLOW CONTROL AND BUFFERING

- **Flow control method 2: use sliding window method as in data link layer. (Cont'd)**
  - Problem: at transport layer, delay is large and variable
    - 1. Fixed window size *m* ➔ Not flexible.
    - 2. Variable delay ➔ lack of ack could be caused by either lost packet, delayed packet, or flow control
      - Hard to set the timer at Tx.
    - 3. Large delay ➔ usually take longer time for Ack to get back to Tx.
      - control is slow.
  - Fixed sliding window is not suitable for transport layer flow control.

ELEMENTS: FLOW CONTROL AND BUFFERING

- **Flow control method 3: credit scheme (variable size sliding window)**
  - Window size is changing based on network condition and buffer size at Rx.
  - Each packet header contains 3 fields (SN, AN, W) (unit: byte)
    - SN: sequence # (byte-wise, the seq# of first byte in packet.)
    - AN: acknowledge #
    - W: window size



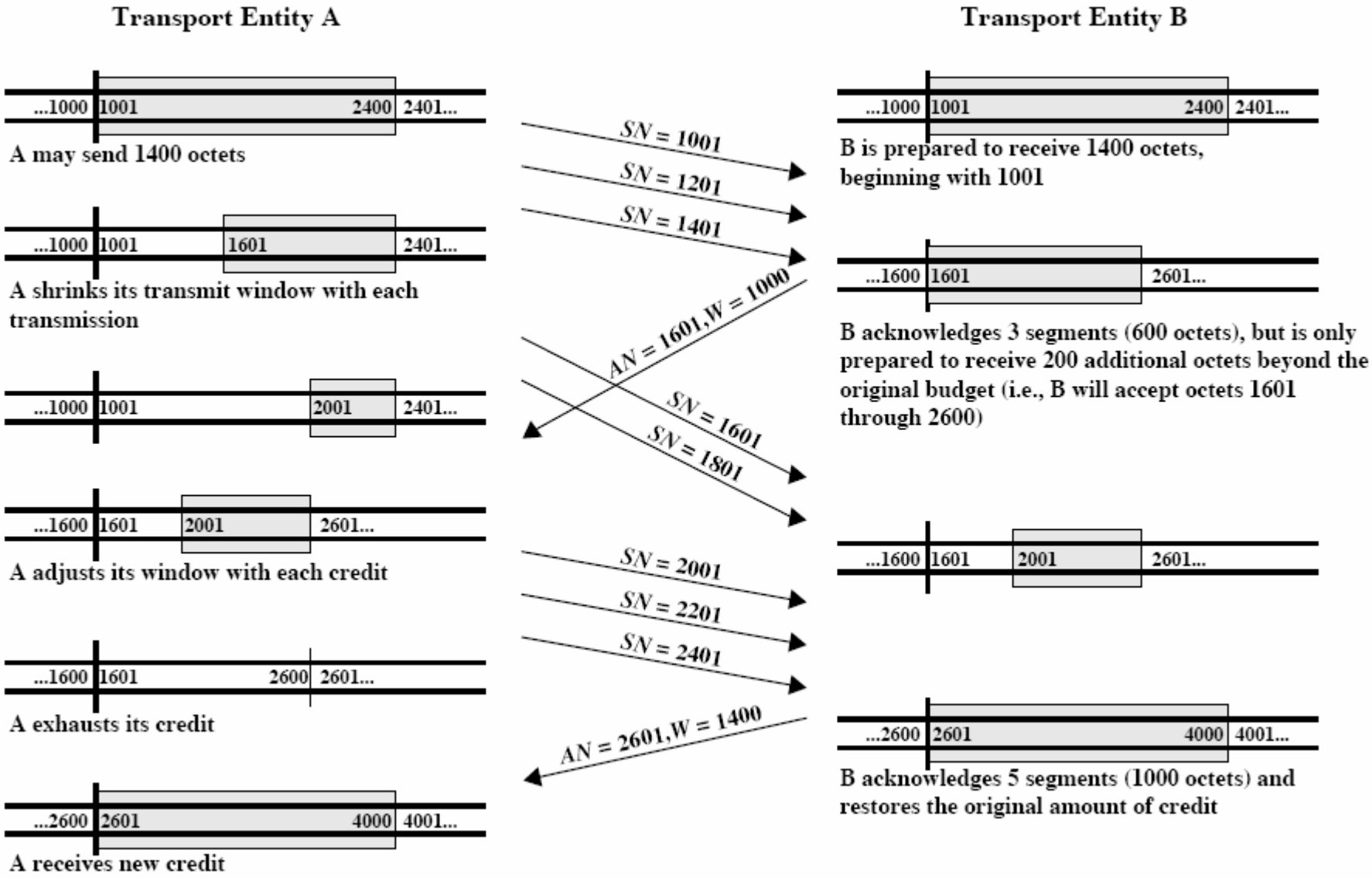
ELEMENTS: FLOW CONTROL AND BUFFERING

- **Flow control method 3: (Cont'd)**
  - E.g. A receives an acknowledgment packet with (SN = k, AN = i, W = j) from B
    - The sequence # of the Ack packet is k
    - All bytes sent by A with SN 0 to i-1 are acknowledged. B is expecting the next packet with SN i.
    - B grants a window of size j to A.
      - A can send bytes from i through i+j-1.
  - Rx sets the window size W = j based on the available buffer size at receiver.
    - The ack and window size is decoupled!
  - Control information (Ack) are periodically sent by the Rx to avoid lost packet.

ELEMENTS: FLOW CONTROL AND BUFFERING

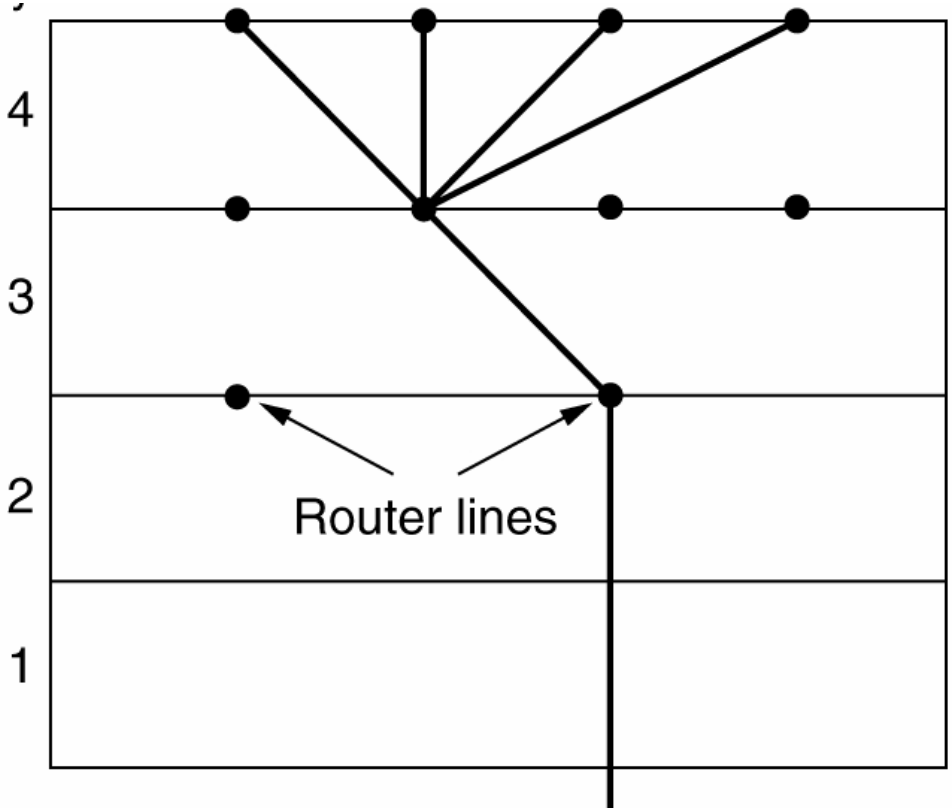
- **Flow control method 3: (Cont'd)**

Packet length: 200 bytes



ELEMENTS: MULTIPLEXING

- **Upward multiplexing**
  - Multiple transport connections use the same network connection.
    - Occurred when only one network address is available at host.

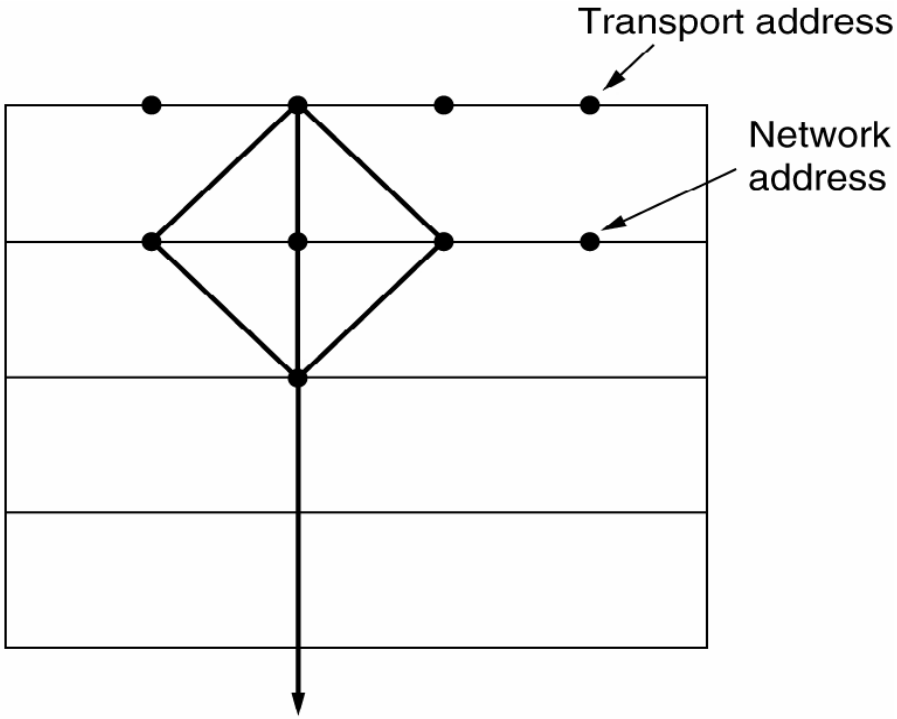


transport address

Network address

ELEMENTS: MULTIPLEXING

- **Downward multiplexing**
  - Distribute the traffic of one transport layer connection onto multiple network connections.
  - E.g. Connection oriented network layer. Each VC allows a maximum data rate of M bps.
    - Transport layer connection wants a data rate of kM bps.



ELEMENTS: CRASH RECOVERY

- **Crash recovery**
  - Routers and network might crash
    - No transport layer on router. Recovery is straightforward.
    - Lost transport PDU (TPDU). Retransmit at transport layer.
  - Hosts crash
    - Crashed host will lost all connection information.
- **Host crash example 1**

  1. server receives packet from client
  2. server passes packet to upper layer (application layer)
  3. Before server sends out Ack, server crashes, then server reboot.
  4. Client learns that server crashed and rebooted.
  5. client got no Ack, retx the same packet.

  Result: duplicate packet at server.

ELEMENTS: CRASH RECOVERY

- **Host crash example 2**

  1. server receives packet from client
  2. server sends Ack to client
  3. Before server passes packet to application layer, server crashes, then server reboots.
  4. Client learns that server crashed and rebooted.
  5. client got Ack, assume packet is successfully delivered to application at server.

  Result: lost packet at server.

Send Ack and pass packet to upper layer cannot be carried out simultaneously!

- **No universal solution.**
  - Must rely on upper layer to resolve the crash on this layer.